FIELD GUIDE · MLOPS

On-Prem AI Model Deployment

— Cheat Sheet —

From staging to production: practical controls, commands, terms, and pitfalls for shipping AI (LLM + RAG) on your own infrastructure — written for the engineer who builds and ships the model, not the platform team.

v1.2 · July 2026 · Kubernetes/OpenShift + vLLM + RAG

SECTION 01

Pre-Production Readiness

Four boxes you must check before any model reaches production.

1.1 · MODEL CARD — MINIMUM REQUIRED FIELDS

- Model name + version + base model/adapter info (e.g. "Mistral-7B-Instruct + LoRA v3"); check that the base model's license (Llama/Mistral/Qwen etc.) allows commercial use
- Adapter/LoRA details + context window + tokenizer must be documented — whichever one changes, behavior changes, so track each separately
- Training/fine-tuning data source, date range, license status
- Evaluation results: the golden Q&A set should be drawn from real (anonymized) production logs — synthetic or academic questions alone miss edge cases; refresh the set as traffic evolves
- Known limitations: hallucination rate (%) measured on the golden set, plus which topics/languages/data types the model is weak on — "it seems to work" is not a sufficient standard
- Owner/team and on-call contact info
- Sign-off chain: who approved it and when — the first document an audit asks for

model_card.yaml EXAMPLE

```yaml
model_name: mistral-7b-instruct-en-lora
version: 2.3.1                    # major.minor.patch
base_model: mistralai/Mistral-7B-Instruct-v0.3
adapter: lora-v3 (r=16, alpha=32, context: 8k)
prompt_template: support-en-v7    # system prompt is versioned separately too
embedding_model: bge-m3 (MIT) · dim=1024
vector_db: qdrant · collection=kb-v14 (142k chunks)
training_data:
  source: internal-support-tickets-2025H2
  date_range: 2025-07-01 / 2025-12-31
  license: internal-proprietary
evaluation:
  test_set: golden-set-v4 (n=850, sampled from prod logs)
  score: 0.88   # acceptance threshold: 0.85
  recall_at_5: 0.91
  hallucination_rate: 0.032
known_limitations:
  - Consistency drops beyond 6K tokens of context
  - Risk of outdated answers on regulatory questions
owner: mlops-team
approved_by: risk-committee
approved_at: 2026-07-02
checksum_sha256: 9f2a1c...        # proof of file integrity
```

1.2 · VERSIONING

- Version model weights + code + config as a TRIPLE together; a checkpoint hash alone is not enough
- The prompt template and system prompt are production artifacts too, versioned like code — even a small system-prompt tweak needs its own version and review, otherwise "same model, different behavior" becomes impossible to debug
- Semantic versioning: major (architecture/base model change) . minor (fine-tune update) . patch (prompt/config fix)
- On-prem model registry: MLflow Model Registry, or a Git-LFS+DVC combo; use stage labels (Staging/Production/Archived)
- If you use RAG: the embedding model version and the knowledge base/index version must be versioned with the same discipline (see 3.4)

1.3 · REPRODUCIBILITY

- Training and inference environments must be pinned via Dockerfile/lockfile; never use a "latest" base image tag — pin by digest
- Record the seed and the hardware type (GPU model, driver, CUDA/cuDNN version) — don't expect bit-identical output on a different GPU
- Periodic reproducibility test: load the same checkpoint on a second node and diff the output against a reference

- Full determinism isn't guaranteed by GPU kernels (cuBLAS, FlashAttention) — different batch sizes, GPUs, or library versions can produce small token-level differences; don't chase this as a "bug", instead watch whether the output quality DISTRIBUTION stays stable

1.4 · DATA & MODEL PROVENANCE

- The training data's source and usage rights/license must be documented (for financial data: compliant with data-classification rules)
- Keep a model lineage: which base model → which fine-tuning steps → which checkpoint
- If you use an open-source/third-party model, have legal/compliance sign off on the license terms (e.g. Llama/Mistral licenses)
- A "Model Bill of Materials": the list of datasets used plus library versions — the model equivalent of an SBOM

> ▸ **FIELD NOTE**
> Shipping to production under a "latest" tag and not knowing which checkpoint is running three months later is a classic mistake. Log the image digest + model version on every deploy, even if it's a single line.

⇄ **CLOUD → ON-PREM DIFFERENCE**

CLOUD:	SageMaker/Vertex Model Registry track lineage automatically.
ON-PREM:	You build and maintain the MLflow+DVC+Git setup yourself — the "registry" isn't a console, it's a service you operate.

1.5 · APPLICATION-LAYER CONTROLS

- Input/output guardrails: defend against prompt injection and jailbreak attempts with a system prompt plus an output filter; check for PII/toxic content leakage across ALL LLM output, not just RAG
- Error/timeout behavior: don't let a model timeout/OOM turn into a silent 500 — define and test a meaningful fallback message plus retry logic
- Secrets in code: API keys/DB credentials must never be hardcoded or committed to the repo; read them from an env var or a mounted secret path (setting up Vault is ops' job — reading it correctly is yours)
- Request/response logging: attach a trace/request ID + model version + a (masked) prompt/response to every request — the code itself should be able to answer "which version produced this" after an incident
- Enforce a rate limit / max_tokens cap in code — per user and overall; don't let a single user or loop monopolize the GPU
- Version your API contract (e.g. /v1/...) so a model or prompt change doesn't break existing callers

SECTION 02

Infrastructure: Model Serving

Choosing and configuring the right serving stack for your model.

2.1 · MODEL SERVING — WHAT TO USE WHEN

- **vLLM** — best for: serving one large LLM at high throughput. Continuous batching + PagedAttention → high throughput, low memory waste, easy OpenAI-compatible API integration
- Start on a single GPU; when you scale to multiple GPUs with tensor-parallel-size, test NCCL/GPU-to-GPU bandwidth as its own separate step
- Tune gpu-memory-utilization carefully if other pods/processes share the box (the default reserves ~90% of VRAM)

TGI (Text Generation Inference) — best for: shipping Hugging Face ecosystem models quickly. Tensor parallelism built in (auto-splits weights across GPUs if the model doesn't fit on one), broad AWQ/GPTQ/bitsandbytes quantization support.

Triton Inference Server — best for: combining LLM + embedding + reranker in one serving layer. Native model ensembles (pre/post-processing chains), Python backend for custom chunking/prompt templates.

Ray Serve — best for: managing the embed → retrieve → rerank → generate chain as code. Scales each step as its own replica (embedding on CPU, LLM on GPU); integrates easily with LangChain/LlamaIndex.

gpu-deployment.yaml + vllm-serve EXAMPLE

```
# --- k8s Deployment excerpt ---
resources:
  limits:   { nvidia.com/gpu: "1", memory: "48Gi" }
  requests: { nvidia.com/gpu: "1", memory: "32Gi" }
nodeSelector: { gpu-type: a100-40gb }
tolerations:
  - { key: nvidia.com/gpu, operator: Exists, effect: NoSchedule }
readinessProbe:
  httpGet: { path: /health, port: 8000 }
  initialDelaySeconds: 60
  periodSeconds: 15

# --- vLLM serve command ---
docker run --gpus '"device=0"' --shm-size=16g \
  -v /data/models/mistral-7b-lora:/model \
  -p 8000:8000 vllm/vllm-openai:v0.6.3 \
  --model /model --quantization awq \
  --tensor-parallel-size 1 \
  --gpu-memory-utilization 0.85 --max-model-len 8192 \
  --served-model-name mistral-7b-en
```

SECTION 03

RAG Architecture — Extra Layers On-Prem

RAG (Retrieval-Augmented Generation) is a whole extra infrastructure layer on top of the LLM — on-prem, you size and monitor every layer yourself.

3.1 · RAG COMPONENTS (END TO END)

- Chain: document source → chunking → embedding model (turns text into vectors) → vector database (storage/search) → retrieval → (optional) re-ranking → context injection into the LLM → response generation
- Every box consumes its own resources: the embedding model wants its own GPU/CPU, the vector DB wants its own RAM/disk — you're capacity-planning a whole chain, not "one model"

3.2 · VECTOR DATABASE SELECTION & SIZING

- On-prem options: Milvus, Qdrant, Weaviate (self-hosted), or add the pgvector extension to an existing PostgreSQL (practical at small/medium scale — no extra system to run)
- Rough sizing: vector dimension (e.g. 1024) × 4 bytes × number of chunks + index overhead (HNSW typically needs ~1.2–1.5× the raw data size in RAM)
- HNSW (fast, in RAM) vs IVF/disk-based (less RAM, a bit slower) — as your document set grows, plan ahead for the risk of not fitting in RAM
- Replication/backup isn't "managed" on-prem — you build the vector DB's own HA/snapshot strategy

3.3 · RETRIEVAL PIPELINE — PRACTICAL SETTINGS

- Chunk size + overlap ripple through model quality; start in the 300–800 token range and test with real questions
- Hybrid search (BM25/keyword + dense/vector) beats pure vector search in most enterprise scenarios — especially for product codes and abbreviations
- Adding a re-ranker (cross-encoder) noticeably improves retrieval quality but costs an extra model plus extra latency — budget for it
- If you CHANGE the embedding model (version or model update), you must re-embed the ENTIRE index — this is a "silent" cost most teams miss the first time

vector-db-collection + embedding-call — EXAMPLE

```
# --- Creating a Qdrant collection (on-prem) ---
collection: enterprise-documents
vectors: { size: 1024, distance: Cosine }
hnsw_config: { m: 16, ef_construct: 128 }
on_disk_payload: true

# --- Embedding service (separate GPU/CPU serving endpoint) ---
POST http://embedding-svc:8080/embed
{"input": ["chunk text..."], "model": "bge-m3"}
```

3.4 · RAG-SPECIFIC VERSIONING & DEPLOYMENT

- Triple versioning: LLM version + embedding model version + knowledge base/index version must be tagged TOGETHER (a change to one requires re-testing compatibility with the others)

- Knowledge base updates are their own pipeline: a source document update should auto-trigger re-ingestion (webhook/cron) — not a manual "if someone remembers" process. Chunking → embedding → upsert must be idempotent; processing the same document twice shouldn't create duplicate vectors
- Blue-green logic applies to the index too: build the new index in a separate collection, run it through tests, then switch retrieval over via an alias swap so you can fall back instantly if quality drops — updating a live index in place is risky

3.5 · RAG SECURITY (ON TOP OF LLM SECURITY)

- The biggest risk: a document the user isn't authorized to see gets retrieved into the LLM's context — that's a data leak; document-level ACLs MUST be reflected in the retrieval query
- Specifically test: "a question from user A should never surface a chunk from user B's document"
- Prompt injection risk: a malicious or tampered document can enter retrieval and hijack the model's behavior via context — keep retrieval sources trusted and scanned

3.6 · RAG MONITORING (ON TOP OF LLM METRICS)

- Retrieval quality: recall@k, MRR (does the right chunk show up in the top-k?) — measure periodically with a golden query set
- Groundedness/faithfulness: is the generated answer actually based on the retrieved context, or is the model hallucinating? An answer that cites no source, or a source not present in retrieval, is the clearest sign of silent quality decay — check automatically and by sampling
- Track the "empty retrieval" rate (no relevant document found) and the context relevance score
- Knowledge base staleness ("index drift"): are newly added documents actually embedded and indexed, how often are they synced — keep a "last sync time" metric

> ▸ FIELD NOTE
> Updating the embedding model and forgetting to re-embed the index is the single most common cause of "why did RAG answer quality drop". Write the embedding model version into the model card.

⇄ CLOUD → ON-PREM DIFFERENCE

CLOUD:	Managed vector search services scale and replicate automatically.
ON-PREM:	You plan the vector DB's sharding/replication/backup, and the fact that the embedding model is its own GPU/CPU resource — RAG isn't "one extra line for the LLM", it's its own capacity-planning item.

SECTION 04

Deployment Strategy

Rollback plan and trigger criteria.

4.1 · ROLLBACK PLAN

- Don't treat rollback as "we'll figure it out if it happens" — rehearse it deliberately in staging and measure how long it takes
- Define the automatic/manual rollback trigger threshold in WRITING and precisely (e.g. "auto-rollback if the 5xx rate exceeds 2%, hallucination rate rises, or p95 exceeds 800ms")
- Critical point: if the embedding model or index changed, rollback must revert the vector index too — rolling back just the LLM container isn't enough
- The previous version's model file + config must always stay reachable ("Archived" state, never deleted)

⇄ CLOUD → ON-PREM DIFFERENCE

CLOUD: Weighted routing/managed load balancers (API Gateway canary, etc.) come out of the box.

ON-PREM: You set up Istio/Argo Rollouts/NGINX yourself — nothing is "out of the box"; test that setup during capacity planning, not during a live incident.

SECTION 05

CI/CD Pipelines

Code and model pipelines flow separately, but hit the same traps in an offline environment.

5.1 · MODEL + CODE PIPELINE SEPARATION

- The code pipeline (serving/app) and the model pipeline (training/fine-tune/eval) are separate but version in sync
- Model pipeline stages: data validation → training/fine-tuning → automated eval (benchmark threshold) → model card generation → registry push
- Base model updates and adapter/LoRA updates should be separate, independently trackable pipelines
- RAG ingestion (document → chunk → embed → index) is a third pipeline with its own trigger (see 3.4)

5.2 · AUTOMATED TESTS

- Unit (serving code) + integration (API contract) + regression (did the new checkpoint drop the score on the old test set?)
- Load/performance test: does the target QPS meet SLA — test on real GPU hardware, not the CI runner's CPU
- Security scan: Trivy/Grype for the image (CVEs); require safetensors for model files (pickle deserialization is unsafe)

SECTION 06

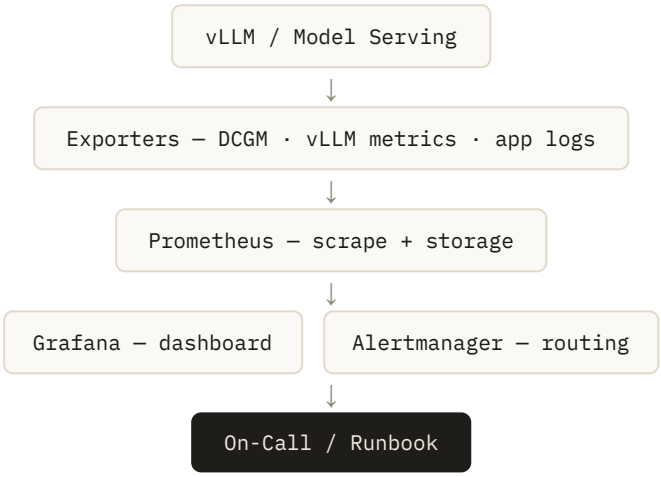
Monitoring & Observability

From latency to drift: what to watch, how, and when.

6.1 · SYSTEM/SERVICE METRICS

- Time to First Token (TTFT) and tokens/sec (generation throughput) — these capture LLM user experience better than p95 latency alone

- Latency p50/p95/p99 (average is NOT enough — tail latency defines LLM user experience), throughput (QPS), error rate (5xx/timeout/OOM) — a spike in OOM count alone should count as a sev1 signal
- GPU metrics: utilization, VRAM usage, temperature, ECC errors via a DCGM exporter — hardware health is now your responsibility
- Export vLLM's own metrics (waiting/running request counts, KV-cache hit rate) to Prometheus
- Track a token counter and GPU-hours — on-prem, this is your equivalent of the cloud's dollar bill, for capacity/energy cost
- If you use RAG, retrieval/groundedness metrics need separate monitoring — they aren't part of LLM metrics (see 3.6)



DRIFT LOOP: Input/Output Logs → PSI/KL Job (periodic) → Drift Alert

6.2 · DATA DRIFT

- Is the input distribution (prompt length, language, topic) drifting from the reference distribution — measure periodically with PSI/KL-divergence
- Example: a new product launch or a regulatory change can suddenly shift the prompt distribution in ways the model never saw

6.3 · MODEL / QUALITY DRIFT

- Output quality: user feedback (thumbs up/down), an automatic LLM-judge score, periodic human sampling review
- Run a golden set (a fixed test question set) after every deploy and on a regular schedule — the score trend catches silent regressions

6.4 · ALERTING

- Alert fatigue is the biggest enemy: only page for things that need action, everything else stays on the dashboard
- Never create an alert without a runbook link — every alert needs a "what to do when this fires" step

prometheus-alert.yaml EXAMPLE

```
- alert: HighInferenceLatencyP99
  expr: histogram_quantile(0.99, rate(vllm_request_latency_seconds_bucket[5m])) > 3
  for: 10m
  labels: { severity: page }
  annotations:
    summary: "p99 latency above 3s for 10 minutes"
    runbook: "https://wiki.internal/runbooks/llm-latency"
```

SECTION 07

Production Launch Best Practices

What to do ahead of time so go-live night has no surprises.

7.1 · PRE-RELEASE CHECKLIST

- ☐ Model card + sign-off complete
- ☐ Load test at target QPS passed in staging
- ☐ Rollback procedure ACTUALLY rehearsed (not just written down)
- ☐ Monitoring dashboards + alert rules ready BEFORE production
- ☐ Runbook current, go-live team roles clear
- ☐ GPU/hardware reservation approved, checked for conflicts with other workloads
- ☐ Guardrail/prompt-injection tests passed
- ☐ Secrets confirmed not hardcoded in code/config

7.2 · FEATURE FLAG / KILL SWITCH

- Put the model call behind a feature flag — you should be able to disable the model without deploying code
- Rehearse the kill switch: actually test "does it really shut off in 10 seconds", don't assume

7.3 · ROLLBACK REHEARSAL

- Do a full rollback rehearsal in staging BEFORE go-live and measure the DURATION — knowing "rollback takes 3 minutes" is a lifesaver mid-crisis

7.4 · GO-LIVE COORDINATION & RUNBOOK

- Roles should be explicit: who runs the deploy command, who watches monitoring, who owns communication
- Runbook contents: step-by-step deploy/rollback commands, key contact info, an escalation chain for "if everything goes wrong"
- Schedule go-live during low-traffic hours, but keep enough people awake and ready

7.5 · POST-PRODUCTION VERIFICATION

- Smoke test: an automated script checking real sample prompts end to end for correct responses
- Health check: readiness/liveness PLUS a real inference call succeeding ("process is up" alone isn't enough)
- First 24–48 hours in hyper-care mode: check metrics more often than usual, keep on-call informed

smoke-test.sh EXAMPLE

```
#!/bin/bash
set -e
RESP=$(curl -sf -X POST http://model-svc:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"mistral-7b-en","messages":[{"role":"user","content":"hello"}]}')
echo "$RESP" | grep -q '"role":"assistant"' \
  && echo "OK: smoke test passed" \
  || (echo "FAIL: smoke test" && exit 1)
```

SECTION 08

Common Mistakes and How to Prevent Them

The mistakes that repeat most often in the field — and how to prevent each one.

MISTAKE	WHY / CONSEQUENCE	PREVENTION
Using the "latest" image tag	No one knows which version is running in production; post-incident analysis becomes impossible	Pin by image digest, require immutable tags
Testing staging on different GPU/hardware than prod	A "worked on my machine" surprise shows up in production	Make hardware parity (GPU type, driver, CUDA) a mandatory checklist item
Never rehearsing rollback	Rollback itself turns out to be broken during a real incident	Run periodic rollback drills and measure the duration
Watching only average (p50) latency	Poor experience for a subset of users stays hidden	Watch p50/p95/p99 together, base alerts on p99
Setting up monitoring only AFTER going to production	A blind spot forms during the first, most critical days	Dashboards + alert rules must be ready BEFORE go-live
Leaving secrets in a ConfigMap/plaintext env var	The first hole found in an audit or pentest	Use Vault / Sealed Secrets / External Secrets Operator
Planning capacity for "average load"	Queueing, timeouts, and lost users at peak hours	Plan for p95 load plus growth headroom; build a queue/rate-limit strategy
Leaving the model card/versioning for last	No answer to "where did this model come from" in an audit	Make it mandatory from day one — no deploy without sign-off
One person deploying from "tribal knowledge"	A crisis becomes unmanageable when that person is out or gone	Write a runbook, get at least two people fluent in the process
Updating the embedding model without re-embedding the index	Retrieval quality silently drops, RAG answers degrade	Manage embedding model + index version together; re-embedding is mandatory on update
Setting chunk size/overlap once and forgetting it	Context becomes irrelevant or fragmented over time	Periodically re-optimize the chunking strategy against a golden set
Treating retrieved content as an "instruction"	Prompt injection can hijack model behavior	Separate data from instructions in the system prompt; mark retrieval output as "data"

APPENDIX

Glossary of Terms

Plain-language explanations of the jargon used in this document — grouped by topic.

GROUP A — MODEL & HARDWARE TERMS

Parameters (B)
The number that indicates a model's "size" (in billions). E.g. 7B = 7 billion parameters. The bigger it is, the more VRAM it needs.

VRAM
The GPU's own memory. Model weights, the KV-cache, and intermediate computations all live here — don't confuse it with system RAM, it's a separate, GPU-only memory pool.

Quantization (fp16/int8/AWQ/GPTQ)
A technique that shrinks the model by representing its weights with fewer bits. Going from fp16 down to int8/4-bit lowers VRAM needs, with a small hit to quality.

KV-Cache
Storing the intermediate "key-value" data the model produces for every token; it grows with context length and concurrent requests, and takes up most of your VRAM.

Context Window
The total number of tokens (input + output) a model can process at once; in RAG, retrieved chunks + the question + the system prompt all have to fit inside it.

Tensor Parallelism
A technique for splitting a single model across multiple GPUs to run in parallel; without a fast GPU-to-GPU link (NVLink), performance suffers.

NVLink / NVSwitch
NVIDIA hardware that connects GPUs directly, much faster than PCIe; it prevents bottlenecks in multi-GPU serving.

MIG (Multi-Instance GPU)
A feature that splits a single NVIDIA GPU into multiple isolated smaller GPUs at the hardware level; useful for running small models/embeddings on a slice of a larger GPU.

Continuous Batching
A serving technique that streams incoming requests together instead of fixed batches, keeping the GPU continuously busy (one of vLLM's core advantages).

PagedAttention
vLLM's technique of managing the KV-cache in pages to reduce memory waste, letting more concurrent requests run in the same VRAM.

LoRA (Low-Rank Adaptation)
A fine-tuning technique that trains small extra layers instead of retraining the whole model; needs far less compute and storage.

Checkpoint
A saved snapshot of a model's weights at a given point during training/fine-tuning; knowing which checkpoint is in production is the basis of versioning.

GROUP B — DEPLOYMENT TERMS

Blue-Green Deployment
A strategy where two full environments (old = blue, new = green) run in parallel, and traffic switches from one to the other instantly.

Canary Deployment
A strategy that opens the new version to a small percentage of traffic first, then ramps it up gradually if metrics look good.

Shadow Deployment
A strategy that feeds the new version real traffic but never returns its response to the user — only logging and comparing it.

Rollback
Reverting to the last known-good version when something breaks in production; its duration and trigger criteria should be defined ahead of time.

Feature Flag / Kill Switch
A toggle that lets you turn a feature (here: the model call) on or off without deploying code.

SLA (Service Level Agreement)
The performance threshold a service commits to meeting (e.g. "p99 latency under 3 seconds").

GROUP C — CI/CD & INFRASTRUCTURE TERMS

Air-Gapped
An environment with no internet connection, physically/logically isolated from external networks; every dependency has to be mirrored in ahead of time.

Artifact Registry
A versioned store for build outputs like images, packages, and models (e.g. Harbor, Nexus, MLflow).

SBOM (Software Bill of Materials)
A full list of a piece of software's dependencies/libraries; the foundation of vulnerability scanning.

Cosign / Image Signing
Cryptographically signing container images to guarantee "this image really came from our pipeline".

CVE
The standard ID assigned to a known security vulnerability; image scans check against these.

RBAC (Role-Based Access Control)
A permission model where access is granted through roles rather than directly to users (e.g. a "deployer" role can deploy but not push to the registry).

NetworkPolicy
A Kubernetes rule set defining which pods/namespaces are allowed to talk to each other.

Vault (Secret Management)
A system that stores sensitive information like API keys and passwords encrypted, with controlled access (e.g. HashiCorp Vault).

GROUP D — MONITORING TERMS

Latency p50/p95/p99
The response time under which 50/95/99 percent of requests complete; p99 shows the slowest user experience and matters more than the average.

Throughput / QPS
The number of requests (queries) a system can handle per second.

Data Drift
When the inputs the model receives (e.g. prompt distribution) gradually diverge from the training/reference data over time.

Model Drift
When a model's output quality degrades over time as usage patterns change.

PSI / KL-Divergence
Statistical methods that numerically measure how much two distributions (e.g. today's prompts vs. reference prompts) differ.

DCGM
NVIDIA's tool for collecting GPU health/usage metrics (utilization, temperature, ECC errors).

Golden Set
A fixed set of test questions with known-good answers, used to monitor model quality over time.

TTFT (Time to First Token)
The time from sending a request to the model producing its first token; the single most important latency metric for LLM user experience.

GROUP E — RAG TERMS

RAG (Retrieval-Augmented Generation)
An architecture where the model first retrieves relevant documents from a data source and uses them as context when generating an answer — helping it use current/factual information it wasn't trained on.

Embedding
Converting text into a numeric vector that represents its meaning; texts with similar meaning end up close together in vector space.

Vector Database
A database that stores embedding vectors and can quickly answer "which vectors are closest to this one" (e.g. Milvus, Qdrant, pgvector).

HNSW
A common index structure vector DBs use to find "nearest neighbors" quickly among millions of vectors; it trades a small amount of accuracy for speed.

Chunking
Splitting long documents into smaller, meaningful pieces for embedding and retrieval.

Hybrid Search
Combining vector (semantic) search with classic keyword (BM25) search.

Re-ranking / Cross-Encoder
Re-sorting the candidates from an initial search with a slower but more accurate model, to surface the best results first.

Groundedness / Faithfulness
A measure of whether a generated answer is actually based on the retrieved source; if it's low, the model is hallucinating.

Top-k
The number of chunks passed to the model as context during retrieval; too few and context is missing, too many and you add irrelevant content plus extra token cost.

Recall@K / Precision@K / MRR
Retrieval quality metrics: recall@k is how often the right answer appears in the top k results, precision@k is how many of the top k are actually relevant, and MRR is the average rank of the correct result (lower is better).

Hallucination
When a model confidently generates information that isn't sourced or is simply wrong; RAG aims to reduce this, but doesn't eliminate it entirely.

Prompt Injection
Malicious text hidden in user input or a retrieved document that tries to override the model's actual instructions (system prompt).